Stoquify Codex Enterprise Operating Workflow

Date: 2026-07-24 Purpose: Combine Codex planning, permissions, reasoning, repository guidance, architecture analysis, verification, review, and context management to evolve Stoquify through proof-driven enterprise delivery.

Central Direction

Stoquify should be evolved through a repeating, evidence-gated delivery loop—not one enormous "make it enterprise-grade" implementation request.

The governing workflow is:

```
Authorized command
→ validated business transaction
→ authoritative domain write
→ immutable/versioned business event
→ accounting/inventory/payroll projection
→ provenance + correlation + content hash
→ assurance evaluation
→ exception/blocker workflow
→ trusted read model
→ reviewable proof pack
```

"Single source of truth" does not mean one giant table or service. It means one authoritative owner for each business fact.

Truth	Authoritative owner
Tenant, identity, and access	Organization/Auth/RBAC control plane
Products, quantities, and valuation inputs	Inventory
Sales, tenders, and terminal activity	POS/Sales
Purchase commitments, receipts, invoices, and supplier payments	Purchasing/AP
Employee, contract, organization, and attendance facts	HRIS
Certified payroll calculations and obligations	Payroll
Journals, balances, periods, and close	Accounting/Ledger
Statutory interpretations	Versioned, expert-approved country packs
Evidence quality and blockers	Assurance—not the source business domain

Current Repository Evidence

- The consolidated July 14 architecture graph covers 6,380 nodes and 9,474 links across services, routes, components, actions, hooks, libraries, types, and Prisma.
- Stoquify has durable foundations for POS offline replay, purchasing/AP, HRIS/payroll, reconciliation, suspense, close assurance, journals, ledger audit events, business events, and workflow-assurance incidents.
- Current static gates report ready for ledger-close truth, payment cash truth, purchasing/AP, offline POS replay, trusted exports, CI configuration, and the Daily Digest cockpit slice.
- Close Assurance is a real accountant trust layer with durable evidence, provenance, hashes, findings, reconciliation certificates, segregation of duties, and close blockers.

Important remaining gaps include:

- no durable database-backed module entitlement source of truth;
- incomplete platform-wide module enforcement;
- HRIS compatibility storage still uses many payroll-named records;
- statutory country-pack evidence is not production-ready until source artifacts are hash-verified and expert-approved;
- production secret readiness is conditional;
- external provider, authority, hardware, deployment, backup/restore, and statutory claims require evidence outside repository tests;
- architecture graphs are older than current July 22–24 work;
- the working tree contains extensive active changes that must be preserved.

Best Command Combination

Repository governance

Do not run `/init` in the current Stoquify root: `AGENTS.md` already exists.

Use `/init` only when a repository or isolated subtree lacks applicable guidance. Durable repository rules belong in `AGENTS.md`; current-task requirements belong in the prompt.

Enterprise discovery

```
/permissions
→ Read Only

/model
→ Strong coding model with High or XHigh reasoning

/goal
→ Establish Stoquify as a proof-driven enterprise operating system

/plan
→ Run the enterprise master prompt
```

Refresh architecture graphs before major dependency decisions when current source changes are newer than the graph artifacts.

Controlled implementation

After one bounded plan is approved:

```
/permissions
→ Workspace or Auto

Reasoning
→ Medium for straightforward implementation
→ High for security, accounting, migrations, concurrency, or cross-domain work
```

Grant additional permissions only for a specifically named action that needs them.

Proof and review

After implementation:

```
/diff
/review
```

Validate progressively:

1. focused tests for the changed invariant;
2. `npm run typecheck`;
3. relevant lint scope;

4. relevant domain gate;
5. cross-domain assurance gate;
6. `npm run verify:repo` when the scope and environment justify the full suite.

A generated `ready` file is not enough on its own. Record the command, exit status, worktree/commit identity, environment, scope, skipped checks, and evidence freshness.

Context management

Run `/compact` only after a coherent, reviewable phase:

- discovery baseline completed;
- plan approved;
- implementation slice completed;
- verification packet recorded; or
- blocker clearly established.

Resume after compaction with:

```
Continue the active Stoquify enterprise goal.

Before acting, reconstruct the current state from:
- the approved plan;
- Git status and diff;
- changed-file ownership;
- completed success criteria;
- executed commands and results;
- generated evidence artifacts;
- remaining blockers and residual risks.

Do not repeat completed work or trust a stale readiness artifact.
```

Enterprise Master Prompt

Run this in Plan mode with Read Only permissions and High/XHigh reasoning:

```
You are the principal enterprise architect, financial-systems engineer,
security reviewer, and proof-governance lead for Stoquify.

MISSION

Inspect the active Stoquify repository thoroughly and design the safest,
highest-leverage sequence for evolving it into a modern, secure, resilient,
multi-tenant enterprise operating system covering:

- POS and offline POS;
- inventory, valuation, counts, adjustments, and transfers;
- purchasing, goods receipt, supplier invoices, and accounts payable;
- sales, customers, payments, cash, and reconciliation;
- accounting, SYSCOHADA/OHADA-aligned ledger operations, and close;
- HRIS People Core, organization, contracts, documents, time, and leave;
- payroll calculation, payment, declarations, and accounting integration;
- statutory country packs and compliance integrations;
- reporting, trusted exports, notifications, and role-based operating cockpits;
- controlled AI agents and workflow automation.

The system must provide one authoritative owner for every business fact and
lead material workflows through reviewable, proof-driven control chains.

THIS IS A DISCOVERY AND PLANNING TURN

Remain read-only. Do not edit files, generate migrations, install packages,
run destructive commands, change external systems, or implement the plan.
Stop after delivering the requested evidence-backed plan.
```

```
REPOSITORY DISCOVERY

1. Read the applicable AGENTS.md files.
2. Inspect Git status and identify modified/untracked files before evaluating
   architecture. Never assume existing changes belong to this task.
3. Read package.json and identify the real validation and release gates.
4. Inspect:
   - graphify-out/ and folder-level architecture graphs;
   - prisma/schema.prisma and recent migrations;
   - services/, actions/, app/, components/, hooks/, lib/, and config/;
   - current architecture, readiness, and execution reports under docs/ and
     what-next/.
5. Check artifact dates against source changes. Treat stale reports and graphs
   as historical evidence, not current truth.
6. Inspect representative implementation and tests for every material claim.
   A roadmap or report alone is not proof of working behavior.

EVIDENCE CLASSIFICATION

Classify every important capability as:

A. PROVEN
   Implemented, test-covered, gate-covered, and supported by current evidence.

B. IMPLEMENTED_NOT_PROVEN
   Code exists, but runtime, integration, environment, or release evidence is
   incomplete.

C. PARTIAL
   A bounded slice works, but the full domain or workflow does not.

D. DOCUMENTED_ONLY
   Present in plans, reports, or UI language without sufficient code evidence.

E. EXTERNAL_DEPENDENCY
   Requires provider, authority, hardware, infrastructure, legal, statutory, or
   expert evidence outside the repository.

F. CONTRADICTED_OR_STALE
   Current source conflicts with an older report, status register, or claim.

Never convert an inference into a verified fact.

STOQUIFY PRODUCT LAWS

1. UI never creates business truth.
2. Each business fact has exactly one authoritative domain owner.
3. Cross-domain consumers use explicit contracts, business events, or certified
   snapshots; they do not silently become alternate writers.
4. HRIS owns mutable people and employment truth.
5. Payroll consumes certified HRIS snapshots and owns payroll calculation,
   payment, and declaration truth.
6. Inventory owns stock-event truth and reproducible projections.
7. Purchasing owns commitments, receipts, invoice matching, and AP workflow.
8. POS owns terminal sale capture, while server replay remains the final legal,
   stock, cash, ledger, and fiscal commit boundary.
9. Accounting owns monetary posting, periods, balances, and close truth.
10. Assurance proves source evidence, freshness, provenance, and blockers; it
    never manufactures business truth.
11. Country packs are versioned, effective-dated, and expert-approved.
    Unsupported statutory automation fails closed.
12. Posted or certified facts are corrected through reversal, supersession, or
    compensating events—not destructive mutation.
13. High-risk workflows require tenant scope, RBAC, fresh authentication,
    maker-checker separation, idempotency, correlation, and audit evidence.
14. Reports disclose whether data is POSTED, OPERATIONAL, ESTIMATED, STALE,
    UNAVAILABLE, or EXTERNALLY_UNVERIFIED.
15. A module cannot claim enterprise readiness from static checks alone.

TARGET PROOF CHAIN
```

```
For every material workflow, assess whether the repository implements:

authorized command
→ validated business rules
→ tenant/module/RBAC/fresh-auth boundary
→ idempotent transaction
→ authoritative domain write
→ business event and outbox
→ downstream projection or posting
→ immutable audit and correlation evidence
→ source/content hash
→ assurance evaluation
→ blocker/exception ownership
→ trusted read model
→ exportable proof packet
→ invalidation when source evidence changes.

Identify every broken or ambiguous link.

DOMAIN ASSESSMENT

For each major domain, report:

- authoritative source of truth;
- allowed writers;
- downstream consumers;
- schemas, services, actions, routes, and jobs involved;
- tenant and authorization boundaries;
- transaction and concurrency guarantees;
- idempotency and duplicate protection;
- correction/reversal behavior;
- business-event and outbox coverage;
- accounting/close implications;
- evidence, provenance, and hash coverage;
- assurance and invalidation coverage;
- observability and recovery;
- test and release-gate coverage;
- production and external dependencies;
- maturity classification and residual risks.

Trace these cross-domain chains:

- POS sale → inventory → tender/payment → ledger → fiscal evidence → close;
- purchase order → receipt → stock → supplier invoice → three-way match
  → AP posting → payment → reconciliation → close;
- HRIS employee/contract/attendance → certified payroll snapshot → payroll run
  → payment/declaration → accounting → close;
- source change after certification → evidence invalidation → reopened blocker
  → reviewed correction → refreshed proof pack.

PRIORITIZATION

Do not propose a rewrite or simultaneous implementation of every module.

Score gaps by:

- financial or legal impact;
- tenant/security exposure;
- source-of-truth ambiguity;
- number of downstream dependencies;
- likelihood of silent data corruption;
- recovery difficulty;
- proof deficiency;
- user/operational value;
- implementation and migration risk.

Select the smallest foundational slice that removes the greatest systemic risk
or unlocks the most downstream capability.
```

```
DELIVERABLE

Produce:

1. Executive verdict.
2. Inspected evidence inventory with artifact freshness.
3. Current capability/maturity matrix.
4. Domain source-of-truth registry.
5. Cross-domain workflow and broken-link analysis.
6. Security, privacy, accounting, compliance, and operational risk register.
7. Contradictions between source, tests, reports, and product claims.
8. Target enterprise architecture.
9. Prioritized program roadmap with dependencies.
10. Recommended next bounded implementation slice.
11. Exact affected files and graph dependency paths for that slice.
12. Data migration/backfill and rollback approach.
13. Test, runtime, browser, security, and release-gate matrix.
14. Required proof artifacts and their storage locations.
15. Explicit GO, GO_WITH_GATES, or NO_GO decision.
16. Questions requiring business, accountant, statutory expert, or
    infrastructure authority.

DONE CONDITIONS

The plan is complete only when:

- facts are separated from inference;
- existing worktree changes are protected;
- each domain has a declared source-of-truth owner;
- cross-domain effects are traced;
- external claims remain explicitly unverified;
- the next slice is bounded and reversible;
- success criteria are measurable;
- verification and rollback are defined;
- no implementation has occurred.

Stop after the plan and wait for approval.
```

Recommended Enterprise Sequence

1. Reconcile the current worktree, status registers, and stale graph evidence.
2. Complete module entitlement persistence and centralized enforcement.
3. Formalize the domain truth registry and write-ownership boundaries.
4. Complete event/outbox/correlation/invalidation coverage across domain writes.
5. Close remaining POS–inventory–payment–ledger–fiscal proof gaps.
6. Strengthen inventory valuation, counts, corrections, and close invalidation.
7. Complete purchasing/AP operational and provider evidence.
8. Stabilize HRIS People Core and migrate away from payroll-named compatibility ownership.
9. Complete HRIS-to-payroll-to-accounting certified snapshot chains.
10. Obtain expert-reviewed, hashed country-pack evidence.
11. Complete production secrets, provider, authority, backup/restore, and deployment evidence.
12. Run controlled pilots before broad enforcement or statutory claims.

Operating Loop

```
Goal
→ Read-only permissions
→ High/XHigh reasoning
→ Refresh architecture evidence
→ Plan and impact analysis
→ Human/accountant/security decision gate
→ Workspace permissions
→ One bounded implementation slice
→ Focused tests and domain gates
→ Cross-domain assurance
→ Diff and review
→ Durable evidence packet
→ Compact
→ Repeat
```